

SOFTWARE TESTING, QUALITY ASSURANCE, AND CI

MVP Test, CI, and Freeze

A 25-30 minute, click-by-click CampusConnect release-candidate lab using Chrome Lighthouse, GitHub Actions, and GitHub

CLASS TIME	MANUAL CHECKS	AUTOMATED GATE	FREEZE TARGET
25-30 min	6	1 workflow	v0.9.0-rc1

LAB OUTCOME

Week 9 assembled the multimodal CampusConnect MVP. Week 10 does not add another feature. You will run one compact quality pass, repair only a release-blocking or severe defect, add one modest CI gate, and freeze the exact accepted commit as release candidate v0.9.0-rc1.

The release rule

A release candidate is earned by evidence. A green GitHub Actions run proves only the deterministic checks encoded in the workflow. The RAG answer, citation support, abstention, voice behavior, muted-video journey, and fallbacks still require the prepared human quality pass.

Evidence lane	What settles the question	CampusConnect example
Exact software behavior	Pass/fail check or CI assertion	app.py compiles, required evidence exists, and no .env file is tracked.
Variable AI behavior	Representative case plus human judgment	A paraphrased password question may use different words but must stay grounded.
Release decision	Severity, retest, and explicit GO / NO-GO	An invented source blocks RC1; a cosmetic spacing issue may be deferred.

DO NOT FORCE THE TAG

If a blocker or severe defect remains after the timeboxed repair, record Release decision: NO-GO and do not create the tag. That is a correct quality decision, not a failed lab. Never lower the severity merely to produce RC1.

What every student produces

Artifact	Required result
Test record	docs/testing/week-10/MVP_TEST_RECORD.md with all six checks, Lighthouse evidence, defect decision, and release gate.
CI workflow	.github/workflows/week10-release-check.yml with one completed run.
Focused repair	One smallest severe-defect repair and retest, or an honest statement that no severe defect was found.
Freeze evidence	v0.9.0-rc1 pre-release if GO; a documented NO-GO with no tag if blocked.

The 30-minute ceiling

Part	Minutes	Stop when...
1. Open the trail	3	The release branch and test record exist
2. Run the MVP pass	9	All six observed results are recorded
3. Run Lighthouse	4	Two scores and one finding are recorded
4. Triage and repair	4	The highest finding is retested or gates NO-GO
5. Add CI	5	The Week 10 workflow completes
6. Freeze	3	RC1 exists only if the release decision is GO

Timebox. Expected finish: about 28 minutes when no severe repair is required. The remaining two minutes are a buffer for GitHub Actions or Lighthouse.

Before the clock starts

- ☐ The corrected Week 9 CampusConnect MVP is committed in your individual repository.
- ☐ LM Studio and app.py are already running; the page opens in Chrome at <http://127.0.0.1:7860>.
- ☐ The page includes the grounded text path plus the Week 7 voice and Week 8 presenter-video paths.
- ☐ You are signed in to GitHub and the repository Actions tab is available.
- ☐ You will use only fictional or instructor-provided test questions - never real credentials or student data.

IF THE APP DOES NOT OPEN

Stop the Week 10 clock. This lab assumes the Week 9 MVP already runs. Ask the instructor for the known-good Week 9 checkpoint rather than spending the testing lab rebuilding the application.

Part 1 - Open the release trail (3 minutes)

TOOL

Use the GitHub website. The release branch isolates the tested candidate from new Week 11 Red Teaming work.

Step 1 - Create the RC1 branch

1. Open <https://github.com/>, sign in, and open your individual CampusConnect semester repository.
2. On the Code page, confirm the branch selector shows main and that the latest Week 9 MVP commit is present.
3. Click the branch selector, type release/week-10-rc1, and choose Create branch: release/week-10-rc1 from main.
4. If the branch menu differs, use [GitHub's official branch guide](#). Return to the Code page and confirm the selector now shows release/week-10-rc1.

Step 2 - Create the prepared test record

1. While the release/week-10-rc1 branch is selected, click Add file, then Create new file.
2. In the filename box, enter docs/testing/week-10/MVP_TEST_RECORD.md. GitHub creates the folders from the forward slashes.
3. Paste Test Record Block A, then paste Block B immediately below it. Replace only your name, date, and environment now. Leave the test fields marked <FILL IN> until you observe the application.
4. Click Commit changes. Use the commit message test: start Week 10 release evidence and commit to the current branch.
5. If needed, compare the screen with [GitHub's official create-file instructions](#).

COPY / PASTE - MVP_TEST_RECORD.md - Block A

Week 10 MVP Test, CI, and Freeze

Student: <FILL IN>
 Date: <FILL IN>
 Branch: release/week-10-rc1
 Local URL: http://127.0.0.1:7860
 Environment: <FILL IN - example: Chrome on macOS; LM Studio local server>

1. Six-test MVP record

#	Quality area	Action and expected result	Observed evidence	Result
1	Supported retrieval	Ask: "How do I reset my CampusConnect password?" The answer uses the approved password-reset passage.	<FILL IN> <FILL IN - PASS or FAIL>	
2	Citation support	Open the displayed source from Test 1. At least one important answer claim is directly supported by the source passage.	<FILL IN> <FILL IN - PASS or FAIL>	
3	Unsupported query	Ask: "What are the Thanksgiving dining hall hours?" The system states its IT-support limit and does not invent hours, a source, phone number, or office.	<FILL IN> <FILL IN - PASS or FAIL>	

COPY / PASTE - MVP_TEST_RECORD.md - Block B (paste directly after Block A)

4	Voice and text fallback	Use the supported answer. If voice works, its meaning matches the visible grounded answer. If voice is unavailable, the text, source, and a clear Continue in text or retry path remain.	<FILL IN> <FILL IN - PASS or FAIL>	
5	Video accessibility	Mute the tab and skip the video. AI disclosure and transcript remain visible, and the chatbot is reachable by keyboard.	<FILL IN> <FILL IN - PASS or FAIL>	
6	Latency and recovery	Time one supported question. Class target: grounded text in 5 seconds or less. If slower, the question is preserved and progress, retry, or text fallback is clear.	<FILL IN - seconds plus behavior> <FILL IN - PASS or FAIL>	

2. Lighthouse and manual accessibility evidence

Performance score: <FILL IN>
 Accessibility score: <FILL IN>
 One important Lighthouse finding or "none listed": <FILL IN>
 Keyboard plus muted-video result: <FILL IN>

3. Defect decision and retest

Highest-severity finding: <FILL IN OR NONE>
 Severity: <FILL IN - BLOCKER, SEVERE, MAJOR, MINOR, or NONE>
 Smallest repair made: <FILL IN OR "No repair required">
 Focused retest result: <FILL IN>

4. Release gate

Release decision: <FILL IN - GO or NO-GO>
 Decision reason: <FILL IN>
 GitHub Actions run: <PASTE RUN URL>
 Release tag if GO: v0.9.0-rc1

5. Engineering and AI Use Log

What I tested myself: <FILL IN>
 What remained human-reviewed rather than automated: <FILL IN>
 Known limitation carried into Week 11 Red Teaming: <FILL IN>

Part 2 - Run the six-test MVP pass (9 minutes)

EVIDENCE

Use one or two plain sentences per observed result. Record what happened, not what the system was supposed to do.

Step 3 - Run the six tests in order

1. Open the running CampusConnect page in Chrome. Keep the GitHub MVP_TEST_RECORD.md editor open in a second tab.
2. Test 1 - Supported retrieval. Ask: How do I reset my CampusConnect password? Record which approved passage or topic the answer used. Do not grade the prose style; grade whether the answer stays within the retrieved support information.
3. Test 2 - Citation support. Open the source shown for Test 1 and locate the passage supporting one important claim. PASS only when the source actually supports that claim. A visible but unrelated link is a FAIL.
4. Test 3 - Unsupported query. Ask: What are the Thanksgiving dining hall hours? PASS when CampusConnect states its IT-support boundary and avoids inventing hours, a source, a phone number, or an office.
5. Test 4 - Voice and fallback. Use the supported password answer. If voice is available, listen long enough to confirm the spoken meaning matches the visible answer. If voice is unavailable or quota-limited, PASS only if the text, source, and a clear continue-or-retry path remain.
6. Test 5 - Video accessibility. Mute the Chrome tab, do not play the presenter video, and press Tab until the question control is reached. PASS when the AI disclosure and transcript remain visible and the chat task can be completed without sound or video.
7. Test 6 - Latency and recovery. Use a phone timer or stopwatch for one supported question. Record the seconds until grounded text is visible. The class target is 5 seconds or less. If it is slower, PASS only when input is preserved and progress, retry, or a usable fallback is clear.
8. Return to MVP_TEST_RECORD.md, click the pencil icon, replace every test-row <FILL IN> with a short observation and PASS or FAIL, then click Commit changes with the message test: record six MVP checks.

Weak evidence	Strong evidence
Citation passed.	The displayed password-reset source explicitly supported the portal step in the answer; no unsupported expiration claim appeared.
Voice did not work.	ElevenLabs was unavailable, but the answer text, source link, typed question, and Continue in text path remained visible.

Weak evidence	Strong evidence
The app was kind of slow.	Grounded text appeared in 7.2 seconds; the question remained visible and a progress state appeared, so the recovery contract passed.

Part 3 - Run one Lighthouse pass (4 minutes)

TOOL

Use Chrome DevTools Lighthouse against the local CampusConnect page. The score is a signal; the muted and keyboard test remains required human evidence.

Step 4 - Audit the local page

1. Keep the CampusConnect page open. Open [Chrome's official Lighthouse guide](#) in another tab if you want a visual reference for the current panel.
2. Return to CampusConnect. Open Chrome DevTools: right-click the page and choose Inspect, or press Option+Command+I on macOS / Control+Shift+I on Windows.
3. Select the Lighthouse panel. If it is hidden, click the double-arrow More tabs button or DevTools menu > More tools > Lighthouse.
4. Choose Navigation mode, Desktop device, and select only Performance and Accessibility. Click Analyze page load and wait for the report.
5. Record the Performance score, Accessibility score, and one important finding. Do not claim that a score of 100 proves accessibility.
6. Use [Chrome's accessibility reference](#) if Lighthouse identifies an unnamed control, contrast problem, or structure issue. Then add the Test 5 keyboard-and-muted result.
7. Edit MVP_TEST_RECORD.md, replace every Lighthouse-section <FILL IN>, and commit with the message test: add Lighthouse evidence.

INTERPRET SCORES CAREFULLY

Lighthouse results are influenced by the computer, extensions, stored browser state, and local workload. Record the environment and the individual finding. Do not compare two students' raw performance scores as if they came from the same controlled test machine.

Part 4 - Triage and repair (4 minutes)

DECISION

Fix only the highest release-blocking or severe finding. Lower-priority polish belongs in the backlog, not in the release freeze.

Step 5 - Make one release decision

1. Review the six tests and Lighthouse finding. Choose the highest severity: BLOCKER, SEVERE, MAJOR, MINOR, or NONE.
2. Treat fabricated facts or sources, loss of the text fallback, an unusable core keyboard path, exposed secrets, or a broken supported journey as BLOCKER or SEVERE. A cosmetic alignment issue is usually MAJOR or MINOR when the task remains usable.
3. If the highest finding is BLOCKER or SEVERE, make the smallest repair that restores the already-defined contract. Examples: restore the Week 8 transcript; keep the answer and citation visible when voice fails; tighten unsupported-query abstention; correct the source mapping.
4. Rerun only the failed test. Commit a focused change such as fix: restore text fallback after voice failure. Do not add a new feature or refactor unrelated code.
5. Edit the defect and release-gate sections of MVP_TEST_RECORD.md. Use Release decision: GO only if no blocker or severe defect remains. Otherwise use Release decision: NO-GO.
6. Replace every remaining <FILL IN> marker, including the Engineering and AI Use Log. For the GitHub Actions run field, temporarily enter Pending - workflow added in Step 6. Commit with the message test: complete release decision.

QUICK-REPAIR BOUNDARY

If the severe repair cannot be completed and retested within four minutes, stop. Record NO-GO, leave the candidate untagged, and carry the defect to a focused repair session. A rushed unverified fix is not release evidence.

Part 5 - Add one modest CI gate (5 minutes)

TOOL

GitHub Actions will check syntax, required release evidence, exact secret hygiene, and the explicit GO decision. It will not call LM Studio, ElevenLabs, or HeyGen.

Step 6 - Create and run the workflow

1. On GitHub, confirm release/week-10-rc1 is selected. Click Add file > Create new file.
2. Enter .github/workflows/week10-release-check.yml as the filename. Paste the complete workflow below exactly as shown.
3. Click Commit changes. Use the message ci: add Week 10 release check and commit to release/week-10-rc1.
4. Open the repository Actions tab. GitHub's [Actions quickstart](#) shows the same workflow-run area if this is your first time using it.
5. Open Week 10 Release Check and wait for the newest run. Expand a red step, read its exact message, correct only that issue, and commit. A new run starts automatically.
6. When the run is green, copy its browser URL. Edit MVP_TEST_RECORD.md, replace Pending - workflow added in Step 6 with the run URL, and commit. Confirm the new run is also green.
7. For the current Python workflow pattern, see [GitHub's official Python CI guide](#). For the least-privilege permissions line, see the official [workflow syntax reference](#).

COPY / PASTE - [.github/workflows/week10-release-check.yml](#)

```

name: Week 10 Release Check
on: [push, pull_request]
permissions:
  contents: read
jobs:
  release-check:
    runs-on: ubuntu-latest
    steps:
      - name: Check out repository
        uses: actions/checkout@v6
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - name: Compile the CampusConnect app
        run: |
          test -f app.py
          test -f requirements.txt
          python -m py_compile app.py
      - name: Verify release evidence and secret hygiene
        run: |
          test -f docs/testing/week-10/MVP_TEST_RECORD.md
          if git ls-files | grep -Eq '^(|/)\.env$'; then
            echo "A tracked .env file is not allowed."
            exit 1
          fi
          if grep -q '<FILL IN>' docs/testing/week-10/MVP_TEST_RECORD.md; then
            echo "Complete every Week 10 evidence field."
            exit 1
          fi
          grep -q '^Release decision: GO$' docs/testing/week-10/MVP_TEST_RECORD.md

```

If this step is red...	Meaning	Smallest repair
Compile the CampusConnect app	app.py or requirements.txt is missing, or app.py has a syntax error.	Restore the expected file or correct the reported syntax line.
Verify release evidence	The record is missing, a tracked .env exists, a marker remains, or the decision is not GO.	Complete the record, untrack .env, or preserve NO-GO and do not tag.

The workflow deliberately grants read-only repository access and does not use model, voice, video, or personal API credentials. A file named .env.example is acceptable only when it contains variable names and no secret values.

Part 6 - Freeze the release candidate (3 minutes)

GATE

Create RC1 only when the test record says GO and the latest workflow run is green.

Step 7 - Tag the tested commit

1. Confirm both conditions: MVP_TEST_RECORD.md says exactly Release decision: GO, and the latest Week 10 Release Check run is green.
2. On the repository Code page, click Releases at the right side, then click Draft a new release.
3. Open Choose a tag, type v0.9.0-rc1, and choose Create new tag. In Target, select release/week-10-rc1 so the tag points to the tested commit.
4. Enter CampusConnect v0.9.0-rc1 as the release title. Paste the release notes block below and replace the final limitation placeholder.
5. Select This is a pre-release. Do not mark RC1 as the latest production release. Click Publish release.
6. GitHub's current [release instructions](#) show the Choose a tag, Target, pre-release, and Publish release controls.
7. If the release decision is NO-GO or CI remains red, stop before creating the tag. Submit the test record and workflow run as correct gate evidence.

COPY / PASTE - RC1 release notes

CampusConnect v0.9.0-rc1

- Integrated MVP: local RAG, visible citations, optional voice, optional presenter video, and text fallback
- Manual evidence: six Week 10 quality checks completed
- Accessibility evidence: Lighthouse plus keyboard and muted-video review
- Release gate: GitHub Actions passed on the tagged commit
- Severe defects: repaired and retested, or none found
- Known limitation for Week 11 Red Teaming: <REPLACE WITH ONE HONEST LIMITATION>

Submission checklist

- ☐ The release/week-10-rc1 branch contains the completed MVP_TEST_RECORD.md.
- ☐ All six quality areas have observed evidence and PASS / FAIL results.
- ☐ Lighthouse scores, one finding, and the keyboard-plus-muted result are recorded.
- ☐ The highest defect has a severity, smallest repair or no-repair statement, and focused retest.

- ☐ The latest Week 10 Release Check run is linked and interpreted correctly.
- ☐ v0.9.0-rc1 exists only for a green, GO candidate; an unresolved candidate correctly remains NO-GO and untagged.

Grading - 100 points

Evidence	Points
Six-test MVP record with concise observed evidence	30
Lighthouse plus manual keyboard and muted-video review	15
Correct severity, focused repair decision, and retest	20
GitHub Actions workflow and interpreted run	20
Evidence-backed GO tag or correctly enforced NO-GO	15

FULL CREDIT IS COMPATIBLE WITH NO-GO

Quality assurance is not theater. A well-supported NO-GO that correctly blocks the tag can earn the same release-decision credit as a valid GO. The evidence must explain the blocker and the next repair.

Bridge to Week 11 - Red Teaming

Week 10 established ordinary quality evidence and, when earned, a stable RC1 target. Week 11 changes posture. The Red Team will try prompt injection, privacy pressure, unsupported requests, misuse, source manipulation, and failure stress against that exact candidate. Preserve the six Week 10 cases as the regression baseline: every Red Team repair must keep ordinary behavior working.

8. Week 10 asks: Does CampusConnect behave as designed under representative conditions?
9. Week 11 asks: How can a determined or creative user violate those assumptions?
10. The RC1 tag lets the team compare before and after without guessing which code was tested.